COMP3308 Assignment 2 Part 2: Report

Student 1: 520027556
Student 2: 530141790
Student 3: 510032494

1.Introduction

This study primarily investigates the performance of several machine learning classification algorithms on a heart failure prediction dataset. We have implemented custom versions of the K-nearest neighbours (KNN) and decision tree (DT) classifiers in Python, as well as improved versions known as KNN+ and DT+. Furthermore, we have compared the performance of these algorithms with various classifiers available in Weka, including Naive Bayes, Support Vector Machines, Random Forests and ensemble learning methods.

As heart failure is a serious condition, early prediction can help improve patient outcomes and survival rates. Machine learning techniques can assist doctors by identifying patterns in medical data and supporting clinical decision-making. By comparing different classification algorithms, we can gain a better understanding of which methods are best suited to medical prediction tasks, and how the choice of algorithm affects predictive performance.

2.Data

This study utilises the 'Heart Failure' dataset (heart.csv). The dataset contains 273 patient records and 12 attributes, covering both clinical and demographic information. These attributes include age, anaemia, creatine phosphokinase (CPK), diabetes, ejection fraction, hypertension, platelets, serum creatinine, serum sodium, gender and smoking status. The categorical attribute indicates whether the patient is alive or deceased.

As the raw numerical data has been discretised into intervals or categories prior to classification, most attributes are categorical. The dataset exhibits moderate class imbalance, with more 'survival' instances than 'death' instances. Consequently, 10-fold hierarchical cross-validation is employed to ensure that each fold retains a class distribution roughly equivalent to that of the original dataset.

3. The KNN+ and DT+ Algorithms

KNN+:

The original K-Nearest Neighbours algorithm classifies test samples by selecting the k nearest training samples and using majority voting to determine the final class. In the standard implementation, each neighbour contributes equally to the prediction, regardless of its distance from the test sample.

The main improvement in KNN+ lies in the introduction of a distance-based weighting mechanism. Neighbours are no longer assigned equal weights; instead, greater weights are allocated to those closer to the test sample, as per the following formula:

Weight $= \frac{1}{distance+1}$

This improvement is based on the assumption that samples closer in distance are typically more similar to the test instance and should therefore exert a stronger influence on the final prediction. The addition of 1 in the denominator is intended to prevent division by zero when the distance is zero. As all attributes in the dataset are categorical, the Hamming distance is used as the distance metric. The distance value increases by 1

whenever two samples differ on a particular attribute.
During the classification process, weighted votes are cast for the ‘Death’ and ‘Survival’ classes respectively, and these are summed. The class with the higher total weight is selected as the predicted outcome. In the event of a tie, the ‘Death’ class is selected in accordance with the task specifications.

DT+:
The original decision tree classifier uses information gain to select attributes. Although information gain is effective for measuring the reduction in entropy following a split, it may favour attributes with many distinct values, which can sometimes result in poor generalisation of the tree. To address this issue, gain ratio is used in place of information gain for attribute selection; this is known as DT+.
Split Information = $-\sum p_i p_i$
Gain Ratio = $\frac{information\ gain}{split\ information}$
The purpose of this modification is to reduce bias towards attributes with many possible values, improve fairness in attribute selection during tree construction, and enhance generalisation ability. Tree growth ceases under the following conditions: all examples in a subtree belong to the same class; all examples have the same attribute value but belong to different classes; or there are no attributes available for further splitting.
When a classification relationship occurs, the ‘dead’ class is selected. Furthermore, if an unseen attribute value is encountered during testing, the majority class stored in the current node is returned.

(See Note 1 for the full code)

4. Results and discussion
All classifiers were evaluated using 10-fold hierarchical cross-validation. Accuracy was used as the primary evaluation metric, as it provides an overall reflection of predictive performance on the heart failure dataset. Under the same evaluation settings, a custom Python implementation was compared with the Weka classifier.

Table 1: Results of Custom Python Classifiers

Classifier	Average Accuracy
MyKNN	65.56%
MyKNN+	65.54%
MyDT	63.77%
MyDT+	58.57%

The experimental results show that the performance of the KNN and KNN+ classifiers is very similar. MyKNN achieved an average accuracy of 65.56%, whilst MyKNN+ achieved 65.54%.
The weighted voting employed in KNN+ neither significantly improved nor reduced performance. This suggests that the distances between neighbouring samples were already relatively close, and therefore the weighting had a limited impact on the final prediction.

The decision tree classifiers performed slightly less well overall. MyDT achieved an accuracy of 63.77%, whilst MyDT+ achieved 58.57%.
Although it was expected that DT+ would improve the quality of feature selection through the information gain ratio, this modification did not increase the overall accuracy on this dataset. This may be because the dataset is relatively small and has limited feature diversity, suggesting that bias in information gain is not a major issue in practical

applications.

Table 2: Results of Weka Classifiers

Classifier	Average Accuracy
ZeroR	67.03%
OneR	71.79%
KNN	65.56%
Naive Bayes (NB)	71.06%
MLP	63.74%
SVM (SMO)	72.53%
DT-unpruned	65.93%
DT-pruned	72.16%
Bagging	69.60%
AdaBoost	67.40%
Random Forest (RF)	68.13%

The results show that the Support Vector Machine achieved the highest accuracy at 72.53%, followed closely by the pruned decision tree at 72.16% and the OneR classifier at 71.79%. These classifiers significantly outperformed the custom Python implementation.

ZeroR achieved an accuracy of 67.03% by consistently predicting the majority class. This result suggests that the dataset is somewhat imbalanced, with more ‘survival’ instances than ‘death’ instances. Consequently, classifiers must achieve a performance of over 67% to demonstrate meaningful predictive capability.

Unusually, the simple OneR classifier achieved highly competitive performance. As OneR constructs rules using only a single attribute, this result indicates that some individual attributes in the dataset contain strong predictive information.

The Weka implementation of KNN achieved an accuracy of 65.56%, which is very similar to the custom Python implementation. This suggests that the fundamental behaviour of custom KNN classifiers is generally correct and consistent with the standard KNN algorithm. However, the performance of the KNN classifier was inferior to that of several other classifiers, possibly because KNN relies heavily on local similarity between instances. As the dataset was discretised into classification ranges, many patients may have exhibited similar attribute patterns.

Naive Bayes also achieved an accuracy of 71.06%. This may be because many attributes in the discretised dataset approximately satisfy the conditional independence assumption used by Naive Bayes.

The Multi-Layer Perceptron (MLP) achieved an accuracy of only 63.74%, which is lower than many simpler methods. As neural network models typically require larger datasets and more parameter tuning to perform well, the limited dataset size may have reduced their effectiveness.

The strong performance of the SVM indicates that the dataset contains patterns that can be effectively separated using margin-based classification. It is well known that SVM methods perform well on relatively small datasets and are generally effective at handling high-dimensional decision boundaries.

Our custom decision tree achieved an accuracy of 63.77%. By comparison, Weka’s unpruned J48 model achieved 65.93%, whilst the pruned version significantly

outperformed all other models with an accuracy of 72.16%.
The performance of the implemented version falls short of Weka's benchmark unpruned tree, suggesting that there may be room for improvement in the custom algorithm regarding splitting criteria, the handling of classification/discrete attributes, or the stopping conditions.
The pruned decision tree also performed significantly better than its unpruned counterpart. The unpruned tree achieved an accuracy of 65.93%, whilst pruning improved this to 72.16%. This suggests that pruning successfully reduced overfitting by removing branches that captured noise rather than useful patterns.

Ensemble methods produced mixed results. Bagging achieved an accuracy of 69.60%, whilst Random Forest reached 68.13%; both represented an improvement over some simpler classifiers but did not outperform SVM or pruned decision trees. AdaBoost achieved 67.40%, only slightly higher than ZeroR. It is possible that the relatively small dataset limited the effectiveness of ensemble learning methods.

Among the four tree-based classifiers, the single pruned decision tree (DT-pruned) performed best with an accuracy of 72.16%, outperforming all three ensemble methods: Bagging (69.60%), Random Forest (68.13%) and AdaBoost (67.40%). This result is unusual, as ensemble methods typically improve the performance of a single tree by reducing variance. This anomaly can be attributed to the small dataset size (only 273 samples) and the limited feature set (12 attributes). Ensemble methods rely on the diversity of their constituent learners to improve predictive performance; with few features, the random subspace sampling in Random Forests is likely to have excluded a key predictive variable, whereas the single decision tree relies heavily on that very variable. Similarly, AdaBoost's sequential reweighting may have overemphasised noisy samples in the small dataset, causing performance to drop to a level close to zero error (ZeroR). Among the ensemble models, Bagging performed best because it resamples the data without restricting the features, thereby preserving the strong dependencies on predictive variables that make the single decision tree model effective.

DT diagrams：**(In Note 2)**

5. Conclusion:
This part mainly compares the performance of custom Python implementations of KNN, KNN+, DT, and DT+ with multiple Weka classifiers on the heart failure prediction dataset, and evaluates them using 10-fold stratified cross-validation. The results show that the Weka SMO classifier achieved the highest accuracy, and the pruned J48 decision tree and OneR classifier also performed well, demonstrating that the margin-based methods and simple rule models have good predictive capabilities on small medical datasets. In contrast, the improvement of the custom algorithm did not bring significant enhancement. The results of MyKNN+ and MyKNN are similar, indicating that the distance-weighting mechanism has limited effect on the discretized category data; the performance of MyDT+ is lower than MyDT, suggesting that gain ratio does not show a significant advantage on this dataset. The experimental results also show that pruning can effectively reduce overfitting and improve the generalization ability of decision trees. Moreover, ensemble methods such as Bagging, AdaBoost, and Random Forest did not exceed a single pruned decision tree, indicating that higher model complexity does not necessarily lead to better performance on small datasets. Future work can further study parameter optimization, different distance measurement methods, category imbalance handling, and more evaluation metrics, such as recall, precision, and F1-score, to more comprehensively evaluate the model's performance in medical prediction.

6. Reflection

530141790:Through this assignment, I gained understanding of the practical applications of machine learning classification algorithms. By implementing KNN and Decision Tree classifiers using Python, I learned how training data, distance calculation, entropy, and information gain are used for prediction. I also understood the importance of stratified cross-validation and how it helps to evaluate model performance more fairly.

Furthermore, an important lesson I learned is that improving an algorithm does not necessarily improve accuracy. While KNN+ and DT+ introduce methods such as weighted voting and gain ratio, their performance is not always superior to the original model. This assignment made me realize that the performance of an algorithm depends heavily on the dataset itself and the characteristics of its features, not just model complexity.

510032494: This assignment helped our group better understand how machine learning algorithms work on real datasets. By implementing KNN, KNN+, DT and DT+ in Python, we learned how classification algorithms make predictions and how different design choices can affect performance.

During the project, we compared our own implementations with Weka classifiers. The results showed that some improvements, such as weighted KNN and gain ratio decision trees, did not always improve accuracy. This taught us that the effectiveness of an algorithm depends heavily on the characteristics of the dataset.

We also learned the importance of model evaluation. Using 10-fold stratified cross-validation allowed us to test the classifiers more fairly. In addition, the experiments showed that simpler models can sometimes perform better than more complex methods on small datasets.

As a group, we improved our teamwork and communication skills by dividing tasks and discussing experimental results together. Overall, this assignment strengthened our understanding of machine learning concepts and practical implementation.

7. Acknowledgement of AI use

520027556: No AI tools were used in this assignment
530141790: No AI tools were used in this assignment
510032494: No AI tools were used in this assignment

NOTE 1

1.KNN+ AND DT+

```
import math


def classify_knnplus(training_filename, testing_filename, k):
    # read data
    training_data = []
    with open(training_filename, 'r') as f:
        for line in f:
            parts = line.strip().split(',')
            # all parameters of the patient
            features = parts[:-1]
            # the final result
            label = parts[-1]
            training_data.append((features, label))

    testing_data = []
    with open(testing_filename, 'r') as f:
        for line in f:
            parts = line.strip().split(',')
            testing_data.append(parts)

    results = []

    for test in testing_data:
        distances = []

        # calculate distance for training data
        for i in range(len(training_data)):
            train_features = training_data[i][0]
            train_label = training_data[i][1]

            distance = 0
            for j in range(len(test)):
                if test[j] != train_features[j]:
                    distance += 1

            # prioritize distance firstly, then consider numbering
            distances.append((distance, i, train_label))

        # sort
        distances.sort()

        # take the first k neighbors
        k_nearest = distances[:k]

        # ***use distance related weights = 1/(distance+1)***
```

```
            score_died = 0
            score_survived = 0

            for item in k_nearest:
                distance = item[0]
                label = item[2]

                weight = 1 / (distance + 1)

                if label == "died":
                    score_died += weight
                else:
                    score_survived += weight

            # conclusion
            if score_died > score_survived:
                results.append("died")
            elif score_survived > score_died:
                results.append("survived")
            else:
                # tie
                results.append("died")

        return results


    def classify_dtplus(training_filename, testing_filename):

        # read data
        def load_training(filename):
            data = []
            with open(filename, 'r') as f:
                for line in f:
                    parts = line.strip().split(',')
                    features = parts[:-1]
                    label = parts[-1]
                    data.append((features, label))
            return data

        def load_testing(filename):
            data = []
            with open(filename, 'r') as f:
                for line in f:
                    parts = line.strip().split(',')
                    data.append(parts)
            return data

        training_data = load_training(training_filename)
        testing_data = load_testing(testing_filename)

        # calculate entropy
        def entropy(data):
            count_died = 0
            count_survived = 0
```

```
    for item in data:
        if item[1] == "died":
            count_died += 1
        else:
            count_survived += 1

    total = count_died + count_survived

    if total == 0:
        return 0

    ent = 0

    for count in [count_died, count_survived]:
        if count != 0:
            p = count / total
            ent -= p * math.log2(p)

    return ent

# calculate information gain
def information_gain(data, attr_index):
    total_entropy = entropy(data)

    # group by attribute value
    subsets = {}
    for item in data:
        key = item[0][attr_index]
        if key not in subsets:
            subsets[key] = []
        subsets[key].append(item)

    # calculate conditional entropy
    weighted_entropy = 0
    total = len(data)

    for subset in subsets.values():
        weighted_entropy += (len(subset) / total) * entropy(subset)

    # ig
    return total_entropy - weighted_entropy

# ***calculate split information***
def split_information(data, attr_index):
    subsets = {}

    for item in data:
        key = item[0][attr_index]
        if key not in subsets:
            subsets[key] = []
        subsets[key].append(item)

    total = len(data)
    si = 0
```

```
    for subset in subsets.values():
        p = len(subset) / total
        if p != 0:
            si -= p * math.log2(p)

    return si


# judgment result
def majority_class(data):
    count_died = 0
    count_survived = 0

    for item in data:
        if item[1] == "died":
            count_died += 1
        else:
            count_survived += 1

    if count_died > count_survived:
        return "died"
    elif count_survived > count_died:
        return "survived"
    # tie
    else:
        return "died"

# build tree
def build_tree(data, attributes):

    # stopping condition
    # check class
    first_label = data[0][1]
    all_same = True
    for item in data:
        if item[1] != first_label:
            all_same = False
            break
    if all_same:
        return first_label

    # check attributes
    def all_same_attributes(data):
        first = data[0][0]
        for item in data:
            if item[0] != first:
                return False
        return True

    if all_same_attributes(data):
        return majority_class(data)

    if len(attributes) == 0:
        return majority_class(data)
```

```
    # ***calculate and use split information***
    # (Equivalent to the "cost-effectiveness" of improving data purity)
    best_attr = attributes[0]
    best_ratio = 0
    first = True

    for attr in attributes:
        gain = information_gain(data, attr)
        si = split_information(data, attr)

        if si == 0:
            ratio = 0
        else:
            ratio = gain / si

        if first or ratio > best_ratio:
            best_ratio = ratio
            best_attr = attr
            first = False

    tree = {
        "attribute": best_attr,
        "children": {},
        "majority": majority_class(data)
    }

    # get value of best_attr
    values = []
    for item in data:
        val = item[0][best_attr]
        if val not in values:
            values.append(val)

    for val in values:
        subset = []
        for item in data:
            if item[0][best_attr] == val:
                subset.append(item)

        # remove used features
        new_attributes = []
        for a in attributes:
            if a != best_attr:
                new_attributes.append(a)

        # dealing with children
        if len(subset) == 0:
            tree["children"][val] = majority_class(data)
        else:
            tree["children"][val] = build_tree(subset, new_attributes)

    return tree

# classification for testing
def classify(tree, example):
```

```
        if type(tree) == str:
            return tree

        attr = tree["attribute"]
        value = example[attr]

        if value in tree["children"]:
            return classify(tree["children"][value], example)
        else:
            return tree["majority"]

    # print tree
    def print_tree(tree, depth=0):
        if type(tree) == str:
            print("  " * depth + "-> " + tree)
            return

        attr = tree["attribute"]
        for val in tree["children"]:
            print("  " * depth + f"[attr {attr} = {val}]")
            print_tree(tree["children"][val], depth + 1)

    # get attributes and construct trees
    num_attributes = len(training_data[0][0])
    attributes = []
    for i in range(num_attributes):
        attributes.append(i)

    tree = build_tree(training_data, attributes)

    # print_tree(default not to print first)

    # processing testing_data
    results = []
    for example in testing_data:
        label = classify(tree, example)
        results.append(label)

       return results
```

NOTE 2

DT diagrams:

```
MY_DT+:
serum_creatinine = high
|   serum_sodium = normal
|   |   high_blood_pressure = no
|   |   |   diabetes = yes
|   |   |   |   sex = M
|   |   |   |   |   age = [61-70]
|   |   |   |   |   |   : died
|   |   |   |   |   age = [40-50]
|   |   |   |   |   |   smoking = yes
```

```
|  |  |  |  |  |  |  : died
|  |  |  |  |  |  smoking = no
|  |  |  |  |  |  |  : survived
|  |  |  |  |  age = 70plus
|  |  |  |  |  |  : died
|  |  |  |  |  age = [51-60]
|  |  |  |  |  |  : survived
|  |  |  |  sex = F
|  |  |  |  |  : died
|  |  |  diabetes = no
|  |  |  |  smoking = no
|  |  |  |  |  platelets = normal
|  |  |  |  |  |  age = [40-50]
|  |  |  |  |  |  |  : died
|  |  |  |  |  |  age = [51-60]
|  |  |  |  |  |  |  anaemia = yes
|  |  |  |  |  |  |  |  sex = M
|  |  |  |  |  |  |  |  |  : died
|  |  |  |  |  |  |  |  sex = F
|  |  |  |  |  |  |  |  |  : survived
|  |  |  |  |  |  |  anaemia = no
|  |  |  |  |  |  |  |  : survived
|  |  |  |  |  |  age = [61-70]
|  |  |  |  |  |  |  ejection_fraction = normal
|  |  |  |  |  |  |  |  anaemia = no
|  |  |  |  |  |  |  |  |  : died
|  |  |  |  |  |  |  |  anaemia = yes
|  |  |  |  |  |  |  |  |  : survived
|  |  |  |  |  |  |  ejection_fraction = low
|  |  |  |  |  |  |  |  : survived
|  |  |  |  |  |  |  ejection_fraction = mildly-low
|  |  |  |  |  |  |  |  : survived
|  |  |  |  |  |  age = 70plus
|  |  |  |  |  |  |  : died
|  |  |  |  |  platelets = low
|  |  |  |  |  |  : survived
|  |  |  |  |  platelets = very-high
|  |  |  |  |  |  : survived
|  |  |  |  smoking = yes
|  |  |  |  |  : survived
|  |  high_blood_pressure = yes
|  |  |  age = [40-50]
|  |  |  |  : died
|  |  |  age = [51-60]
|  |  |  |  platelets = normal
|  |  |  |  |  anaemia = yes
|  |  |  |  |  |  sex = F
|  |  |  |  |  |  |  : died
|  |  |  |  |  |  sex = M
|  |  |  |  |  |  |  : survived
|  |  |  |  |  anaemia = no
|  |  |  |  |  |  : survived
|  |  |  |  platelets = high
|  |  |  |  |  : died
|  |  |  age = [61-70]
```

```
|  |  |  |  : died
|  |  |  age = 70plus
|  |  |  |  : died
|  serum_sodium = low
|  |  platelets = low
|  |  |  : died
|  |  platelets = normal
|  |  |  ejection_fraction = mildly-low
|  |  |  |  diabetes = no
|  |  |  |  |  : died
|  |  |  |  diabetes = yes
|  |  |  |  |  : survived
|  |  |  ejection_fraction = low
|  |  |  |  anaemia = yes
|  |  |  |  |  CPK = normal
|  |  |  |  |  |  : died
|  |  |  |  |  CPK = high
|  |  |  |  |  |  diabetes = yes
|  |  |  |  |  |  |  : died
|  |  |  |  |  |  diabetes = no
|  |  |  |  |  |  |  age = 70plus
|  |  |  |  |  |  |  |  : died
|  |  |  |  |  |  |  age = [61-70]
|  |  |  |  |  |  |  |  : survived
|  |  |  |  anaemia = no
|  |  |  |  |  high_blood_pressure = yes
|  |  |  |  |  |  smoking = no
|  |  |  |  |  |  |  : died
|  |  |  |  |  |  smoking = yes
|  |  |  |  |  |  |  age = [61-70]
|  |  |  |  |  |  |  |  : died
|  |  |  |  |  |  |  age = [40-50]
|  |  |  |  |  |  |  |  : survived
|  |  |  |  |  high_blood_pressure = no
|  |  |  |  |  |  CPK = very-high
|  |  |  |  |  |  |  diabetes = no
|  |  |  |  |  |  |  |  : died
|  |  |  |  |  |  |  diabetes = yes
|  |  |  |  |  |  |  |  : survived
|  |  |  |  |  |  CPK = high
|  |  |  |  |  |  |  diabetes = yes
|  |  |  |  |  |  |  |  age = [61-70]
|  |  |  |  |  |  |  |  |  : died
|  |  |  |  |  |  |  |  age = [51-60]
|  |  |  |  |  |  |  |  |  : survived
|  |  |  |  |  |  |  |  age = 70plus
|  |  |  |  |  |  |  |  |  : survived
|  |  |  |  |  |  |  diabetes = no
|  |  |  |  |  |  |  |  : died
|  |  |  |  |  |  CPK = normal
|  |  |  |  |  |  |  : survived
|  |  |  ejection_fraction = borderline
|  |  |  |  : survived
|  |  |  ejection_fraction = normal
|  |  |  |  : survived
```

```
|   |   platelets = very-high
|   |   |   : died
|   serum_sodium = high
|   |   : died
serum_creatinine = normal
|   ejection_fraction = normal
|   |   serum_sodium = normal
|   |   |   sex = F
|   |   |   |   anaemia = yes
|   |   |   |   |   CPK = normal
|   |   |   |   |   |   diabetes = no
|   |   |   |   |   |   |   : died
|   |   |   |   |   |   diabetes = yes
|   |   |   |   |   |   |   : survived
|   |   |   |   |   CPK = high
|   |   |   |   |   |   : died
|   |   |   |   anaemia = no
|   |   |   |   |   : survived
|   |   |   sex = M
|   |   |   |   : survived
|   |   serum_sodium = low
|   |   |   anaemia = yes
|   |   |   |   : died
|   |   |   anaemia = no
|   |   |   |   age = 70plus
|   |   |   |   |   : died
|   |   |   |   age = [40-50]
|   |   |   |   |   : survived
|   |   |   |   age = [51-60]
|   |   |   |   |   : survived
|   ejection_fraction = low
|   |   sex = F
|   |   |   smoking = no
|   |   |   |   serum_sodium = normal
|   |   |   |   |   platelets = normal
|   |   |   |   |   |   CPK = normal
|   |   |   |   |   |   |   diabetes = no
|   |   |   |   |   |   |   |   high_blood_pressure = no
|   |   |   |   |   |   |   |   |   : died
|   |   |   |   |   |   |   |   high_blood_pressure = yes
|   |   |   |   |   |   |   |   |   : survived
|   |   |   |   |   |   |   diabetes = yes
|   |   |   |   |   |   |   |   : survived
|   |   |   |   |   |   CPK = high
|   |   |   |   |   |   |   diabetes = yes
|   |   |   |   |   |   |   |   anaemia = yes
|   |   |   |   |   |   |   |   |   : died
|   |   |   |   |   |   |   |   anaemia = no
|   |   |   |   |   |   |   |   |   age = 70plus
|   |   |   |   |   |   |   |   |   |   : died
|   |   |   |   |   |   |   |   |   age = [40-50]
|   |   |   |   |   |   |   |   |   |   : survived
|   |   |   |   |   |   |   |   |   age = [51-60]
|   |   |   |   |   |   |   |   |   |   : survived
|   |   |   |   |   |   |   |   |   age = [61-70]
```

```
|  |  |  |  |  |  |  |  |  |  : survived
|  |  |  |  |  |  |  diabetes = no
|  |  |  |  |  |  |  |  : survived
|  |  |  |  |  |  CPK = very-high
|  |  |  |  |  |  |  : survived
|  |  |  |  |  platelets = low
|  |  |  |  |  |  age = [40-50]
|  |  |  |  |  |  |  : died
|  |  |  |  |  |  age = [51-60]
|  |  |  |  |  |  |  : survived
|  |  |  |  |  platelets = high
|  |  |  |  |  |  : survived
|  |  |  |  |  platelets = very-high
|  |  |  |  |  |  : survived
|  |  |  |  serum_sodium = low
|  |  |  |  |  : survived
|  |  |  smoking = yes
|  |  |  |  : died
|  |  sex = M
|  |  |  platelets = normal
|  |  |  |  CPK = high
|  |  |  |  |  diabetes = yes
|  |  |  |  |  |  anaemia = no
|  |  |  |  |  |  |  smoking = yes
|  |  |  |  |  |  |  |  age = [61-70]
|  |  |  |  |  |  |  |  |  high_blood_pressure = no
|  |  |  |  |  |  |  |  |  |  : died
|  |  |  |  |  |  |  |  |  high_blood_pressure = yes
|  |  |  |  |  |  |  |  |  |  : survived
|  |  |  |  |  |  |  |  age = 70plus
|  |  |  |  |  |  |  |  |  : died
|  |  |  |  |  |  |  smoking = no
|  |  |  |  |  |  |  |  : survived
|  |  |  |  |  |  anaemia = yes
|  |  |  |  |  |  |  : survived
|  |  |  |  |  diabetes = no
|  |  |  |  |  |  age = [61-70]
|  |  |  |  |  |  |  anaemia = no
|  |  |  |  |  |  |  |  high_blood_pressure = no
|  |  |  |  |  |  |  |  |  smoking = yes
|  |  |  |  |  |  |  |  |  |  : died
|  |  |  |  |  |  |  |  |  smoking = no
|  |  |  |  |  |  |  |  |  |  : survived
|  |  |  |  |  |  |  |  high_blood_pressure = yes
|  |  |  |  |  |  |  |  |  : survived
|  |  |  |  |  |  |  anaemia = yes
|  |  |  |  |  |  |  |  high_blood_pressure = yes
|  |  |  |  |  |  |  |  |  : died
|  |  |  |  |  |  |  |  high_blood_pressure = no
|  |  |  |  |  |  |  |  |  : survived
|  |  |  |  |  |  age = [40-50]
|  |  |  |  |  |  |  anaemia = no
|  |  |  |  |  |  |  |  smoking = no
|  |  |  |  |  |  |  |  |  : died
|  |  |  |  |  |  |  smoking = yes
```

```
|  |  |  |  |  |  |  |  |  high_blood_pressure = yes
|  |  |  |  |  |  |  |  |  |  : died
|  |  |  |  |  |  |  |  |  high_blood_pressure = no
|  |  |  |  |  |  |  |  |  |  : survived
|  |  |  |  |  |  |  |  anaemia = yes
|  |  |  |  |  |  |  |  |  : survived
|  |  |  |  |  |  |  age = 70plus
|  |  |  |  |  |  |  |  high_blood_pressure = no
|  |  |  |  |  |  |  |  |  serum_sodium = low
|  |  |  |  |  |  |  |  |  |  : died
|  |  |  |  |  |  |  |  |  serum_sodium = normal
|  |  |  |  |  |  |  |  |  |  : survived
|  |  |  |  |  |  |  |  high_blood_pressure = yes
|  |  |  |  |  |  |  |  |  : survived
|  |  |  |  |  CPK = normal
|  |  |  |  |  |  age = [61-70]
|  |  |  |  |  |  |  : survived
|  |  |  |  |  |  age = [40-50]
|  |  |  |  |  |  |  high_blood_pressure = yes
|  |  |  |  |  |  |  |  : died
|  |  |  |  |  |  |  high_blood_pressure = no
|  |  |  |  |  |  |  |  : survived
|  |  |  |  |  |  age = [51-60]
|  |  |  |  |  |  |  anaemia = yes
|  |  |  |  |  |  |  |  high_blood_pressure = no
|  |  |  |  |  |  |  |  |  serum_sodium = normal
|  |  |  |  |  |  |  |  |  |  : died
|  |  |  |  |  |  |  |  |  serum_sodium = low
|  |  |  |  |  |  |  |  |  |  : survived
|  |  |  |  |  |  |  |  high_blood_pressure = yes
|  |  |  |  |  |  |  |  |  : survived
|  |  |  |  |  |  |  anaemia = no
|  |  |  |  |  |  |  |  : survived
|  |  |  |  |  |  age = 70plus
|  |  |  |  |  |  |  : died
|  |  |  |  |  CPK = very-high
|  |  |  |  |  |  smoking = no
|  |  |  |  |  |  |  anaemia = no
|  |  |  |  |  |  |  |  high_blood_pressure = no
|  |  |  |  |  |  |  |  |  : died
|  |  |  |  |  |  |  |  high_blood_pressure = yes
|  |  |  |  |  |  |  |  |  serum_sodium = low
|  |  |  |  |  |  |  |  |  |  : died
|  |  |  |  |  |  |  |  |  serum_sodium = normal
|  |  |  |  |  |  |  |  |  |  : survived
|  |  |  |  |  |  |  anaemia = yes
|  |  |  |  |  |  |  |  : survived
|  |  |  |  |  |  smoking = yes
|  |  |  |  |  |  |  : survived
|  |  |  |  |  CPK = severely-high
|  |  |  |  |  |  smoking = no
|  |  |  |  |  |  |  : died
|  |  |  |  |  |  smoking = yes
|  |  |  |  |  |  |  age = 70plus
|  |  |  |  |  |  |  |  : died
```

```
|  |  |  |  |  |  age = [40-50]
|  |  |  |  |  |  |  : survived
|  |  |  platelets = low
|  |  |  |  anaemia = yes
|  |  |  |  |  age = [40-50]
|  |  |  |  |  |  : died
|  |  |  |  |  age = 70plus
|  |  |  |  |  |  : died
|  |  |  |  |  age = [51-60]
|  |  |  |  |  |  : survived
|  |  |  |  anaemia = no
|  |  |  |  |  : survived
|  |  |  platelets = high
|  |  |  |  : died
|  |  |  platelets = very-high
|  |  |  |  : survived
|  ejection_fraction = borderline
|  |  smoking = no
|  |  |  diabetes = no
|  |  |  |  CPK = high
|  |  |  |  |  anaemia = no
|  |  |  |  |  |  : died
|  |  |  |  |  anaemia = yes
|  |  |  |  |  |  platelets = low
|  |  |  |  |  |  |  : died
|  |  |  |  |  |  platelets = normal
|  |  |  |  |  |  |  : survived
|  |  |  |  CPK = normal
|  |  |  |  |  : survived
|  |  |  |  CPK = very-high
|  |  |  |  |  : survived
|  |  |  diabetes = yes
|  |  |  |  anaemia = no
|  |  |  |  |  age = [61-70]
|  |  |  |  |  |  : died
|  |  |  |  |  age = 70plus
|  |  |  |  |  |  : survived
|  |  |  |  anaemia = yes
|  |  |  |  |  : died
|  |  smoking = yes
|  |  |  : survived
|  ejection_fraction = mildly-low
|  |  high_blood_pressure = yes
|  |  |  smoking = yes
|  |  |  |  diabetes = yes
|  |  |  |  |  : died
|  |  |  |  diabetes = no
|  |  |  |  |  : survived
|  |  |  smoking = no
|  |  |  |  age = 70plus
|  |  |  |  |  CPK = normal
|  |  |  |  |  |  : died
|  |  |  |  |  CPK = very-high
|  |  |  |  |  |  : survived
|  |  |  |  |  CPK = high
```

```
|  |  |  |  |  |  : survived
|  |  |  |  age = [40-50]
|  |  |  |  |  : survived
|  |  |  |  age = [51-60]
|  |  |  |  |  : survived
|  |  |  |  age = [61-70]
|  |  |  |  |  : survived
|  |  high_blood_pressure = no
|  |  |  : survived
serum_creatinine = low
|  : survived


MY_DT
serum_creatinine = high
|  serum_sodium = normal
|  |  high_blood_pressure = no
|  |  |  diabetes = yes
|  |  |  |  age = [61-70]
|  |  |  |  |  : died
|  |  |  |  age = [40-50]
|  |  |  |  |  smoking = yes
|  |  |  |  |  |  : died
|  |  |  |  |  smoking = no
|  |  |  |  |  |  : survived
|  |  |  |  age = [51-60]
|  |  |  |  |  sex = F
|  |  |  |  |  |  : died
|  |  |  |  |  sex = M
|  |  |  |  |  |  : survived
|  |  |  |  age = 70plus
|  |  |  |  |  : died
|  |  |  diabetes = no
|  |  |  |  smoking = no
|  |  |  |  |  age = [40-50]
|  |  |  |  |  |  : died
|  |  |  |  |  age = [51-60]
|  |  |  |  |  |  anaemia = yes
|  |  |  |  |  |  |  sex = M
|  |  |  |  |  |  |  |  : died
|  |  |  |  |  |  |  sex = F
|  |  |  |  |  |  |  |  : survived
|  |  |  |  |  |  anaemia = no
|  |  |  |  |  |  |  : survived
|  |  |  |  |  age = [61-70]
|  |  |  |  |  |  ejection_fraction = normal
|  |  |  |  |  |  |  anaemia = no
|  |  |  |  |  |  |  |  : died
|  |  |  |  |  |  |  anaemia = yes
|  |  |  |  |  |  |  |  : survived
|  |  |  |  |  |  ejection_fraction = low
|  |  |  |  |  |  |  : survived
|  |  |  |  |  |  ejection_fraction = mildly-low
|  |  |  |  |  |  |  : survived
```

```
|  |  |  |  |  age = 70plus
|  |  |  |  |  |  CPK = high
|  |  |  |  |  |  |  : died
|  |  |  |  |  |  CPK = normal
|  |  |  |  |  |  |  : survived
|  |  |  |  smoking = yes
|  |  |  |  |  : survived
|  |  high_blood_pressure = yes
|  |  |  age = [40-50]
|  |  |  |  : died
|  |  |  age = [51-60]
|  |  |  |  anaemia = yes
|  |  |  |  |  sex = F
|  |  |  |  |  |  : died
|  |  |  |  |  sex = M
|  |  |  |  |  |  : survived
|  |  |  |  anaemia = no
|  |  |  |  |  : survived
|  |  |  age = [61-70]
|  |  |  |  : died
|  |  |  age = 70plus
|  |  |  |  : died
|  serum_sodium = low
|  |  platelets = low
|  |  |  : died
|  |  platelets = normal
|  |  |  ejection_fraction = mildly-low
|  |  |  |  CPK = high
|  |  |  |  |  : died
|  |  |  |  CPK = normal
|  |  |  |  |  : died
|  |  |  |  CPK = very-high
|  |  |  |  |  : survived
|  |  |  ejection_fraction = low
|  |  |  |  age = [40-50]
|  |  |  |  |  sex = F
|  |  |  |  |  |  : died
|  |  |  |  |  sex = M
|  |  |  |  |  |  : survived
|  |  |  |  age = [51-60]
|  |  |  |  |  anaemia = yes
|  |  |  |  |  |  : died
|  |  |  |  |  anaemia = no
|  |  |  |  |  |  diabetes = no
|  |  |  |  |  |  |  : died
|  |  |  |  |  |  diabetes = yes
|  |  |  |  |  |  |  : survived
|  |  |  |  age = [61-70]
|  |  |  |  |  diabetes = yes
|  |  |  |  |  |  : died
|  |  |  |  |  diabetes = no
|  |  |  |  |  |  high_blood_pressure = yes
|  |  |  |  |  |  |  : died
|  |  |  |  |  |  high_blood_pressure = no
|  |  |  |  |  |  |  : survived
```

```
|   |   |   age = 70plus
|   |   |   |   diabetes = no
|   |   |   |   |   : died
|   |   |   |   diabetes = yes
|   |   |   |   |   high_blood_pressure = yes
|   |   |   |   |   |   : died
|   |   |   |   |   high_blood_pressure = no
|   |   |   |   |   |   : survived
|   |   |   ejection_fraction = borderline
|   |   |   |   : survived
|   |   |   ejection_fraction = normal
|   |   |   |   : survived
|   |   platelets = very-high
|   |   |   : died
|   serum_sodium = high
|   |   : died
serum_creatinine = normal
|   ejection_fraction = normal
|   |   age = [61-70]
|   |   |   high_blood_pressure = no
|   |   |   |   diabetes = no
|   |   |   |   |   anaemia = yes
|   |   |   |   |   |   : died
|   |   |   |   |   anaemia = no
|   |   |   |   |   |   : survived
|   |   |   |   diabetes = yes
|   |   |   |   |   : survived
|   |   |   high_blood_pressure = yes
|   |   |   |   : survived
|   |   age = [51-60]
|   |   |   CPK = high
|   |   |   |   diabetes = yes
|   |   |   |   |   : died
|   |   |   |   diabetes = no
|   |   |   |   |   : survived
|   |   |   CPK = normal
|   |   |   |   : survived
|   |   |   CPK = very-high
|   |   |   |   : survived
|   |   age = 70plus
|   |   |   : died
|   |   age = [40-50]
|   |   |   : survived
|   ejection_fraction = low
|   |   age = [61-70]
|   |   |   CPK = normal
|   |   |   |   high_blood_pressure = no
|   |   |   |   |   sex = F
|   |   |   |   |   |   : died
|   |   |   |   |   sex = M
|   |   |   |   |   |   : survived
|   |   |   |   high_blood_pressure = yes
|   |   |   |   |   : survived
|   |   |   CPK = high
|   |   |   |   smoking = yes
```

```
|  |  |  |  |  high_blood_pressure = no
|  |  |  |  |  |  : died
|  |  |  |  |  high_blood_pressure = yes
|  |  |  |  |  |  anaemia = no
|  |  |  |  |  |  |  : survived
|  |  |  |  |  |  anaemia = yes
|  |  |  |  |  |  |  : died
|  |  |  |  smoking = no
|  |  |  |  |  anaemia = yes
|  |  |  |  |  |  diabetes = no
|  |  |  |  |  |  |  high_blood_pressure = yes
|  |  |  |  |  |  |  |  sex = M
|  |  |  |  |  |  |  |  |  : died
|  |  |  |  |  |  |  |  sex = F
|  |  |  |  |  |  |  |  |  : survived
|  |  |  |  |  |  |  high_blood_pressure = no
|  |  |  |  |  |  |  |  : survived
|  |  |  |  |  |  diabetes = yes
|  |  |  |  |  |  |  : survived
|  |  |  |  |  anaemia = no
|  |  |  |  |  |  : survived
|  |  |  CPK = very-high
|  |  |  |  : survived
|  |  age = [40-50]
|  |  |  platelets = normal
|  |  |  |  high_blood_pressure = yes
|  |  |  |  |  CPK = high
|  |  |  |  |  |  diabetes = yes
|  |  |  |  |  |  |  : survived
|  |  |  |  |  |  diabetes = no
|  |  |  |  |  |  |  : died
|  |  |  |  |  CPK = normal
|  |  |  |  |  |  : died
|  |  |  |  |  CPK = severely-high
|  |  |  |  |  |  : died
|  |  |  |  |  CPK = very-high
|  |  |  |  |  |  : survived
|  |  |  |  high_blood_pressure = no
|  |  |  |  |  CPK = high
|  |  |  |  |  |  smoking = no
|  |  |  |  |  |  |  diabetes = no
|  |  |  |  |  |  |  |  anaemia = no
|  |  |  |  |  |  |  |  |  : died
|  |  |  |  |  |  |  |  anaemia = yes
|  |  |  |  |  |  |  |  |  : survived
|  |  |  |  |  |  |  diabetes = yes
|  |  |  |  |  |  |  |  anaemia = yes
|  |  |  |  |  |  |  |  |  : died
|  |  |  |  |  |  |  |  anaemia = no
|  |  |  |  |  |  |  |  |  : survived
|  |  |  |  |  |  smoking = yes
|  |  |  |  |  |  |  : survived
|  |  |  |  |  CPK = very-high
|  |  |  |  |  |  anaemia = no
|  |  |  |  |  |  |  smoking = no
```

```
|  |  |  |  |  |  |  |  : died
|  |  |  |  |  |  |  smoking = yes
|  |  |  |  |  |  |  |  : survived
|  |  |  |  |  |  anaemia = yes
|  |  |  |  |  |  |  : survived
|  |  |  |  |  CPK = normal
|  |  |  |  |  |  : survived
|  |  |  |  |  CPK = severely-high
|  |  |  |  |  |  : survived
|  |  |  platelets = low
|  |  |  |  anaemia = yes
|  |  |  |  |  : died
|  |  |  |  anaemia = no
|  |  |  |  |  : survived
|  |  |  platelets = high
|  |  |  |  : survived
|  |  |  platelets = very-high
|  |  |  |  : survived
|  |  age = [51-60]
|  |  |  platelets = normal
|  |  |  |  CPK = normal
|  |  |  |  |  anaemia = yes
|  |  |  |  |  |  high_blood_pressure = no
|  |  |  |  |  |  |  serum_sodium = normal
|  |  |  |  |  |  |  |  : died
|  |  |  |  |  |  |  serum_sodium = low
|  |  |  |  |  |  |  |  : survived
|  |  |  |  |  |  high_blood_pressure = yes
|  |  |  |  |  |  |  : survived
|  |  |  |  |  anaemia = no
|  |  |  |  |  |  : survived
|  |  |  |  CPK = severely-high
|  |  |  |  |  : died
|  |  |  |  CPK = very-high
|  |  |  |  |  serum_sodium = low
|  |  |  |  |  |  : died
|  |  |  |  |  serum_sodium = normal
|  |  |  |  |  |  high_blood_pressure = no
|  |  |  |  |  |  |  sex = M
|  |  |  |  |  |  |  |  : died
|  |  |  |  |  |  |  sex = F
|  |  |  |  |  |  |  |  : survived
|  |  |  |  |  |  high_blood_pressure = yes
|  |  |  |  |  |  |  : survived
|  |  |  |  CPK = high
|  |  |  |  |  anaemia = yes
|  |  |  |  |  |  : died
|  |  |  |  |  anaemia = no
|  |  |  |  |  |  : survived
|  |  |  platelets = high
|  |  |  |  CPK = high
|  |  |  |  |  : died
|  |  |  |  CPK = normal
|  |  |  |  |  : survived
|  |  |  platelets = low
```

```
|  |  |  |  : survived
|  |  |  platelets = very-high
|  |  |  |  : survived
|  |  age = 70plus
|  |  |  CPK = high
|  |  |  |  diabetes = no
|  |  |  |  |  sex = M
|  |  |  |  |  |  anaemia = yes
|  |  |  |  |  |  |  : died
|  |  |  |  |  |  anaemia = no
|  |  |  |  |  |  |  serum_sodium = low
|  |  |  |  |  |  |  |  high_blood_pressure = no
|  |  |  |  |  |  |  |  |  : died
|  |  |  |  |  |  |  |  high_blood_pressure = yes
|  |  |  |  |  |  |  |  |  : survived
|  |  |  |  |  |  |  serum_sodium = normal
|  |  |  |  |  |  |  |  : survived
|  |  |  |  |  sex = F
|  |  |  |  |  |  : survived
|  |  |  |  diabetes = yes
|  |  |  |  |  anaemia = no
|  |  |  |  |  |  : died
|  |  |  |  |  anaemia = yes
|  |  |  |  |  |  : survived
|  |  |  CPK = normal
|  |  |  |  : died
|  |  |  CPK = severely-high
|  |  |  |  : died
|  |  |  CPK = very-high
|  |  |  |  : survived
|  ejection_fraction = borderline
|  |  diabetes = no
|  |  |  CPK = high
|  |  |  |  platelets = normal
|  |  |  |  |  age = [40-50]
|  |  |  |  |  |  anaemia = no
|  |  |  |  |  |  |  : died
|  |  |  |  |  |  anaemia = yes
|  |  |  |  |  |  |  : survived
|  |  |  |  |  age = [61-70]
|  |  |  |  |  |  : survived
|  |  |  |  |  age = 70plus
|  |  |  |  |  |  : survived
|  |  |  |  platelets = low
|  |  |  |  |  : died
|  |  |  CPK = normal
|  |  |  |  : survived
|  |  |  CPK = very-high
|  |  |  |  : survived
|  |  diabetes = yes
|  |  |  anaemia = no
|  |  |  |  age = [61-70]
|  |  |  |  |  CPK = normal
|  |  |  |  |  |  : died
|  |  |  |  |  CPK = high
```

```
|  |  |  |  |  |  : survived
|  |  |  |  age = 70plus
|  |  |  |  |  : survived
|  |  |  anaemia = yes
|  |  |  |  : died
|  ejection_fraction = mildly-low
|  |  high_blood_pressure = yes
|  |  |  age = [61-70]
|  |  |  |  anaemia = no
|  |  |  |  |  : died
|  |  |  |  anaemia = yes
|  |  |  |  |  : survived
|  |  |  age = 70plus
|  |  |  |  CPK = normal
|  |  |  |  |  : died
|  |  |  |  CPK = very-high
|  |  |  |  |  : survived
|  |  |  |  CPK = high
|  |  |  |  |  : survived
|  |  |  age = [40-50]
|  |  |  |  : survived
|  |  |  age = [51-60]
|  |  |  |  : survived
|  |  high_blood_pressure = no
|  |  |  : survived
serum_creatinine = low
|  : survived
```

3.KNN+ , DT+ and DT update

```
import math


attribute_names = [
    "age",
    "anaemia",
    "CPK",
    "diabetes",
    "ejection_fraction",
    "high_blood_pressure",
    "platelets",
    "serum_creatinine",
    "serum_sodium",
    "sex",
    "smoking"
]
def classify_knnplus(training_filename, testing_filename, k):
    # read data
    training_data = []
    with open(training_filename, 'r') as f:
```

```
        for line in f:
            parts = line.strip().split(',')
            # all parameters of the patient
            features = parts[:-1]
            # the final result
            label = parts[-1]
            training_data.append((features, label))

    testing_data = []
    with open(testing_filename, 'r') as f:

        for line in f:
            parts = line.strip().split(',')
            testing_data.append(parts)

    results = []

    for test in testing_data:
        distances = []

        # calculate distance for training data
        for i in range(len(training_data)):
            train_features = training_data[i][0]
            train_label = training_data[i][1]

            distance = 0
            for j in range(len(test)):
                if test[j] != train_features[j]:
                    distance += 1

                    # prioritize distance firstly, then consider
numbering
            distances.append((distance, i, train_label))

        # sort
        distances.sort()

        # take the first k neighbors
        k_nearest = distances[:k]

        # ***use distance related weights = 1/(distance+1)***
        score_died = 0
        score_survived = 0
```

```
        for item in k_nearest:
            distance = item[0]
            label = item[2]

            weight = 1 / (distance + 1)

            if label == "died":
                score_died += weight
            else:
                score_survived += weight

        # conclusion
        if score_died > score_survived:
            results.append("died")
        elif score_survived > score_died:
            results.append("survived")
        else:
            # tie
            results.append("died")

    return results


def classify_dtplus(training_filename, testing_filename):

    # read data
    def load_training(filename):
        data = []
        with open(filename, 'r') as f:
            next(f)
            for line in f:
                parts = line.strip().split(',')
                features = parts[:-1]
                label = parts[-1]
                data.append((features, label))
        return data

    def load_testing(filename):
        data = []
        with open(filename, 'r') as f:
            next(f)
            for line in f:
```

```
            parts = line.strip().split(',')
            data.append(parts)
    return data

training_data = load_training(training_filename)
testing_data = load_testing(testing_filename)

# calculate entropy
def entropy(data):
    count_died = 0
    count_survived = 0

    for item in data:
        if item[1] == "died":
            count_died += 1
        else:
            count_survived += 1

    total = count_died + count_survived

    if total == 0:
        return 0

    ent = 0

    for count in [count_died, count_survived]:
        if count != 0:
            p = count / total
            ent -= p * math.log2(p)

    return ent

# calculate information gain
def information_gain(data, attr_index):
    total_entropy = entropy(data)

    # group by attribute value
    subsets = {}
    for item in data:
        key = item[0][attr_index]
        if key not in subsets:
            subsets[key] = []
        subsets[key].append(item)
```

```
        # calculate conditional entropy
        weighted_entropy = 0
        total = len(data)

        for subset in subsets.values():
                    weighted_entropy += (len(subset) / total) *
entropy(subset)

        # ig
        return total_entropy - weighted_entropy

    # ***calculate split information***
    def split_information(data, attr_index):
        subsets = {}

        for item in data:
            key = item[0][attr_index]
            if key not in subsets:
                subsets[key] = []
            subsets[key].append(item)

        total = len(data)
        si = 0

        for subset in subsets.values():
            p = len(subset) / total
            if p != 0:
                si -= p * math.log2(p)

        return si


    # judgment result
    def majority_class(data):
        count_died = 0
        count_survived = 0

        for item in data:
            if item[1] == "died":
                count_died += 1
            else:
                count_survived += 1
```

```
        if count_died > count_survived:
            return "died"
        elif count_survived > count_died:
            return "survived"
        # tie
        else:
            return "died"

    # build tree
    def build_tree(data, attributes):

        # stopping condition
        # check class
        first_label = data[0][1]
        all_same = True
        for item in data:
            if item[1] != first_label:
                all_same = False
                break
        if all_same:
            return first_label

        # check attributes
        def all_same_attributes(data):
            first = data[0][0]
            for item in data:
                if item[0] != first:
                    return False
            return True

        if all_same_attributes(data):
            return majority_class(data)

        if len(attributes) == 0:
            return majority_class(data)

        # ***calculate and use split information***
          # (Equivalent to the "cost-effectiveness" of improving
data purity)
        best_attr = attributes[0]
        best_ratio = 0
        first = True
```

```
    for attr in attributes:
        gain = information_gain(data, attr)
        si = split_information(data, attr)

        if si == 0:
            ratio = 0
        else:
            ratio = gain / si

        if first or ratio > best_ratio:
            best_ratio = ratio
            best_attr = attr
            first = False

    tree = {
        "attribute": best_attr,
        "children": {},
        "majority": majority_class(data)
    }

    # get value of best_attr
    values = []
    for item in data:
        val = item[0][best_attr]
        if val not in values:
            values.append(val)

    for val in values:
        subset = []
        for item in data:
            if item[0][best_attr] == val:
                subset.append(item)

        # remove used features
        new_attributes = []
        for a in attributes:
            if a != best_attr:
                new_attributes.append(a)

        # dealing with children
        if len(subset) == 0:
            tree["children"][val] = majority_class(data)
```

```
            else:
                        tree["children"][val] = build_tree(subset,
new_attributes)

        return tree

    # classification for testing
    def classify(tree, example):
        if type(tree) == str:
            return tree

        attr = tree["attribute"]
        value = example[attr]

        if value in tree["children"]:
            return classify(tree["children"][value], example)
        else:
            return tree["majority"]

    # print tree
    #def print_tree(tree, depth=0):
        #if type(tree) == str:
            #print("  " * depth + "-> " + tree)
            #return

        #attr = tree["attribute"]
        #for val in tree["children"]:
            #print("  " * depth + f"[attr {attr} = {val}]")
            #print_tree(tree["children"][val], depth + 1)
    def print_tree(tree, depth=0):
        if type(tree) == str:
            print("|   " * depth + ": " + tree)
            return

        attr = tree["attribute"]
        attr_name = attribute_names[attr]

        for val in tree["children"]:
            print("|   " * depth + f"{attr_name} = {val}")
            print_tree(tree["children"][val], depth + 1)
    # get attributes and construct trees
    num_attributes = len(training_data[0][0])
    attributes = []
```

```
    for i in range(num_attributes):
        attributes.append(i)

    tree = build_tree(training_data, attributes)

    # print_tree(default not to print first)
    print_tree(tree)

    # processing testing_data
    results = []
    for example in testing_data:
        label = classify(tree, example)
        results.append(label)

    return results


def classify_dt(training_filename, testing_filename):

    # read data
    def load_training(filename):
        data = []
        with open(filename, 'r') as f:
            next(f)
            for line in f:
                parts = line.strip().split(',')
                features = parts[:-1]
                label = parts[-1]
                data.append((features, label))
        return data

    def load_testing(filename):
        data = []
        with open(filename, 'r') as f:
            next(f)
            for line in f:
                parts = line.strip().split(',')
                data.append(parts)
        return data

    training_data = load_training(training_filename)
    testing_data = load_testing(testing_filename)
```

```
    # calculate entropy
    def entropy(data):
        count_died = 0
        count_survived = 0

        for item in data:
            if item[1] == "died":
                count_died += 1
            else:
                count_survived += 1

        total = count_died + count_survived

        if total == 0:
            return 0

        ent = 0

        for count in [count_died, count_survived]:
            if count != 0:
                p = count / total
                ent -= p * math.log2(p)

        return ent

    # calculate information gain
    def information_gain(data, attr_index):
        total_entropy = entropy(data)

        # group by attribute value
        subsets = {}
        for item in data:
            key = item[0][attr_index]
            if key not in subsets:
                subsets[key] = []
            subsets[key].append(item)

        # calculate conditional entropy
        weighted_entropy = 0
        total = len(data)

        for subset in subsets.values():
                    weighted_entropy += (len(subset) / total) *
```

```
entropy(subset)

        # ig
        return total_entropy - weighted_entropy

    # ***calculate split information***
    def split_information(data, attr_index):
        subsets = {}

        for item in data:
            key = item[0][attr_index]
            if key not in subsets:
                subsets[key] = []
            subsets[key].append(item)

        total = len(data)
        si = 0

        for subset in subsets.values():
            p = len(subset) / total
            if p != 0:
                si -= p * math.log2(p)

        return si


    # judgment result
    def majority_class(data):
        count_died = 0
        count_survived = 0

        for item in data:
            if item[1] == "died":
                count_died += 1
            else:
                count_survived += 1

        if count_died > count_survived:
            return "died"
        elif count_survived > count_died:
            return "survived"
        # tie
        else:
```

```
            return "died"

    # build tree
    def build_tree(data, attributes):

        # stopping condition
        # check class
        first_label = data[0][1]
        all_same = True
        for item in data:
            if item[1] != first_label:
                all_same = False
                break
        if all_same:
            return first_label

        # check attributes
        def all_same_attributes(data):
            first = data[0][0]
            for item in data:
                if item[0] != first:
                    return False
            return True

        if all_same_attributes(data):
            return majority_class(data)

        if len(attributes) == 0:
            return majority_class(data)

        # ***calculate and use split information***
          # (Equivalent to the "cost-effectiveness" of improving
data purity)
         # MyDT: choose the attribute with the highest information
gain
        best_attr = attributes[0]
        best_gain = information_gain(data, best_attr)

        for attr in attributes:
            gain = information_gain(data, attr)

            if gain > best_gain:
                best_gain = gain
```

```
                best_attr = attr

        tree = {
            "attribute": best_attr,
            "children": {},
            "majority": majority_class(data)
        }

        # get value of best_attr
        values = []
        for item in data:
            val = item[0][best_attr]
            if val not in values:
                values.append(val)

        for val in values:
            subset = []
            for item in data:
                if item[0][best_attr] == val:
                    subset.append(item)

            # remove used features
            new_attributes = []
            for a in attributes:
                if a != best_attr:
                    new_attributes.append(a)

            # dealing with children
            if len(subset) == 0:
                tree["children"][val] = majority_class(data)
            else:
                        tree["children"][val] = build_tree(subset,
new_attributes)

        return tree

    # classification for testing
    def classify(tree, example):
        if type(tree) == str:
            return tree

        attr = tree["attribute"]
        value = example[attr]
```

```
    if value in tree["children"]:
        return classify(tree["children"][value], example)
    else:
        return tree["majority"]

# print tree
#def print_tree(tree, depth=0):
    #if type(tree) == str:
        #print("  " * depth + "-> " + tree)
        #return

    #attr = tree["attribute"]
    #for val in tree["children"]:
        #print("  " * depth + f"[attr {attr} = {val}]")
        #print_tree(tree["children"][val], depth + 1)
def print_tree(tree, depth=0):
    if type(tree) == str:
        print("|   " * depth + ": " + tree)
        return

    attr = tree["attribute"]
    attr_name = attribute_names[attr]

    for val in tree["children"]:
        print("|   " * depth + f"{attr_name} = {val}")
        print_tree(tree["children"][val], depth + 1)
# get attributes and construct trees
num_attributes = len(training_data[0][0])
attributes = []
for i in range(num_attributes):
    attributes.append(i)

tree = build_tree(training_data, attributes)

# print_tree(default not to print first)
print_tree(tree)

# processing testing_data
results = []
for example in testing_data:
    label = classify(tree, example)
    results.append(label)
```

```
    return results

#classify_dtplus("heart-failure.csv", "heart-failure.csv")

classify_dt("heart-failure.csv", "heart-failure.csv")
```